



HACETTEPE UNIVERSITY
DEPARTMENT OF ELECTRICAL & ELECTRONICS ENGINEERING

ELE432

ADVANCED DIGITAL DESIGN

Preliminary Work - 3

29/04/2026

NAME AND SURNAME: RAHİME ESRA KARADENİZ

STUDENT ID: 21947325

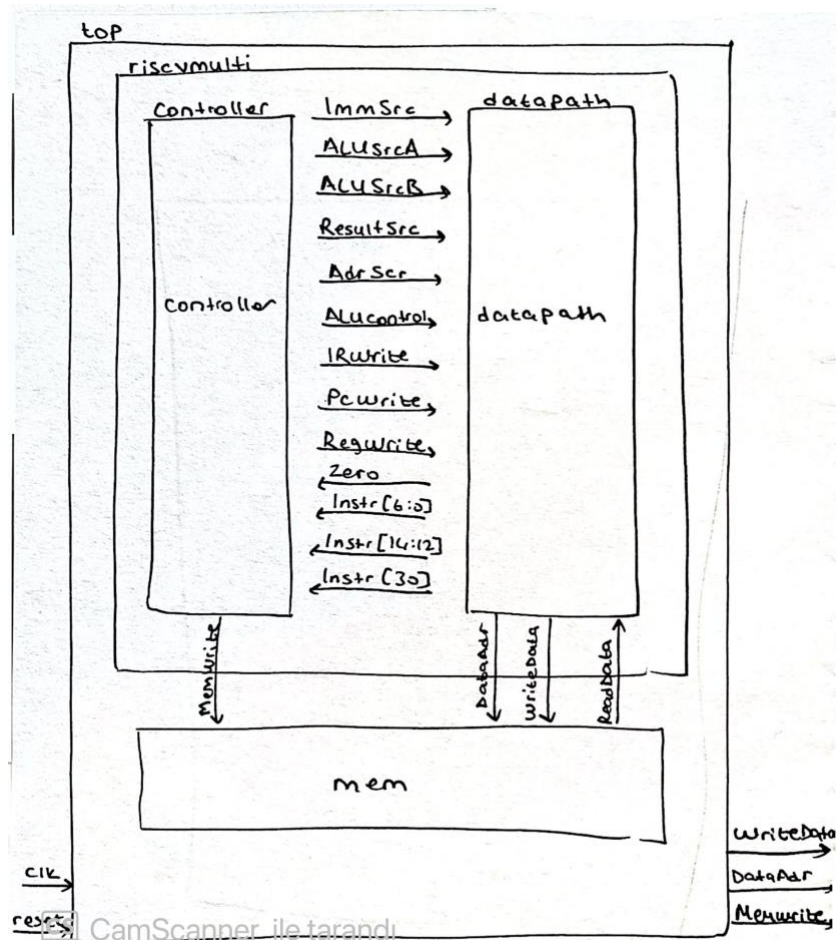
1. Time Spent (+ Github Repo Link)

I spent around 8 to 10 hours on this lab. Most of the time went into understanding how the multicycle datapath differs from the single-cycle one, writing the FSM state transitions correctly and getting all the control signals right for each state. Once the FSM was correct the rest came together fairly quickly.

<https://github.com/esibleksii/ELE432---Preliminary-work-3.git>

2. Block Hierarchy Diagram

The figure below shows the module hierarchy of the multicycle processor. The top module contains two submodules: riscvmulti (the processor) and mem (the unified instruction and data memory). Inside riscvmulti there are two submodules: controller (which contains the mainfsm, aludec and instrdec) and datapath. The signals between each block are listed in the table below.



Connection	Signal(s)	Description
top → riscvmulti	clk, reset	Clock and reset inputs
riscvmulti → top	MemWrite, DataAdr, WriteData	Tapped memory signals for testbench
riscvmulti ↔ mem	MemWrite (we), DataAdr (a), WriteData (wd), ReadData (rd)	Unified memory interface
controller → datapath	ImmSrc, ALUSrcA, ALUSrcB, ResultSrc, AdrSrc, ALUControl, IRWrite, PCWrite, RegWrite, MemWrite	All control signals
datapath → controller	Instr[6:0], Instr[14:12], Instr[30], Zero	Instruction fields and ALU zero flag
datapath → mem	DataAdr (a), WriteData (wd)	Address and write data
mem → datapath	ReadData (rd)	Data read from memory

3. SystemVerilog Code

I wrote a hierarchical SystemVerilog description following the module declaration given in the lab manual. All modules are in a single file for ease of compilation. The key design decisions are listed below.

Top-level module: top contains riscvmulti (the processor) and a unified mem module. The mem module uses \$readmemh("memfile.txt", RAM) at startup.

Controller: The mainfsm implements 11 states (S0 to S10). PCWrite is computed outside the FSM as (Branch AND Zero) OR PCUpdate so that the FSM only needs to output Branch and PCUpdate as separate signals.

Datapath: The OldPC register is enabled by IRWrite (same enable as the instruction register) so it captures the current PC at the same moment the instruction is latched. The ALUOut register

stores the ALU result from each cycle and the result mux chooses between ALUOut, Data and the live ALUResult depending on ResultSrc.

JAL handling: In state S9, the ALU recomputes OldPC+4 (the return address) while the PC is simultaneously loaded from ALUOut (which holds OldPC+ImmExt from the decode cycle). State S7 then writes the new ALUOut (return address) to the destination register.

The module declaration as required:

```
module top(input logic clk, reset,
           output logic [31:0] WriteData, DataAdr,
           output logic      MemWrite);
```

First 80 lines of the full SystemVerilog file are shown below. The complete file is submitted with this report.

```
//=====
// ELE432 - Lab 3 : Multicycle RISC-V Processor
// Reference : Harris & Harris, RISC-V Edition, Ch. 7
// Top module : top(clk, reset, WriteData, DataAdr, MemWrite)
//=====

//----- TOP : RISC-V + unified memory -----
module top(input logic      clk, reset,
           output logic [31:0] WriteData, DataAdr,
           output logic      MemWrite);

    logic [31:0] ReadData;

    // processor (controller + datapath)
    riscvmulti rvmulti(clk, reset, MemWrite, DataAdr, WriteData, ReadData);

    // unified instruction & data memory
    mem      memunit(clk, MemWrite, DataAdr, WriteData, ReadData);
endmodule

//----- Unified instruction / data memory -----
module mem(input logic      clk, we,
           input logic [31:0] a, wd,
           output logic [31:0] rd);

    logic [31:0] RAM[63:0];
```

```

initial
    $readmemh("memfile.txt", RAM);

assign rd = RAM[a[31:2]];                                // word aligned read

always_ff @(posedge clk)
    if (we) RAM[a[31:2]] <= wd;
endmodule

//----- RISC-V multicycle processor -----
module riscvmulti(input  logic      clk, reset,
                  output logic      MemWrite,
                  output logic [31:0] Adr, WriteData,
                  input  logic [31:0] ReadData);

    logic [31:0] Instr;
    logic      Zero;
    logic      PCWrite, RegWrite, IRWrite, AdrSrc;
    logic [1:0] ResultSrc, ALUSrcA, ALUSrcB, ImmSrc;
    logic [2:0] ALUControl;

    controller c(clk, reset,
                 Instr[6:0], Instr[14:12], Instr[30], Zero,
                 ImmSrc, ALUSrcA, ALUSrcB, ResultSrc, AdrSrc,
                 ALUControl, IRWrite, PCWrite, RegWrite, MemWrite);

    datapath dp(clk, reset, ResultSrc, ALUSrcA, ALUSrcB, AdrSrc,
                ImmSrc, ALUControl, IRWrite, PCWrite, RegWrite,
                ReadData, Adr, WriteData, Instr, Zero);
endmodule

//----- Controller (FSM + ALU dec + Imm dec) -----
module controller(input  logic      clk, reset,
                  input  logic [6:0] op,
                  input  logic [2:0] funct3,
                  input  logic      funct7b5,
                  input  logic      Zero,
                  output logic [1:0] ImmSrc,
                  output logic [1:0] ALUSrcA, ALUSrcB,
                  output logic [1:0] ResultSrc,
                  output logic      AdrSrc,
                  output logic [2:0] ALUControl,
                  output logic      IRWrite, PCWrite,

```

```

        output logic      RegWrite, MemWrite);

logic [1:0] ALUOp;
logic      Branch, PCUpdate;

mainfsm fsm(clk, reset, op,
            ALUSrcA, ALUSrcB, ResultSrc, AdrSrc,
            IRWrite, PCUpdate, Branch,
            RegWrite, MemWrite, ALUOp);

aludec ad (op[5], funct3, funct7b5, ALUOp, ALUControl);

instrdec id(op, ImmSrc);

// PC is updated when an unconditional update occurs OR a taken branch
assign PCWrite = (Branch & Zero) | PCUpdate;
endmodule

//----- Main FSM -----
module mainfsm(input logic      clk, reset,
               input logic [6:0] op,
               output logic [1:0] ALUSrcA, ALUSrcB,
               output logic [1:0] ResultSrc,
               output logic      AdrSrc,
               output logic      IRWrite, PCUpdate, Branch,
               output logic      RegWrite, MemWrite,
               output logic [1:0] ALUOp);

typedef enum logic [3:0] {
    S0_FETCH      = 4'd0,
    S1_DECODE     = 4'd1,
    S2_MEMADR     = 4'd2,
    S3_MEMREAD    = 4'd3,
    S4_MEMWB      = 4'd4,
    S5_MEMWRITE   = 4'd5,
    S6_EXECUTER   = 4'd6,
    S7_ALUWB      = 4'd7,
    S8_EXECUTEI   = 4'd8,
    S9_JAL        = 4'd9,
    S10_BEQ       = 4'd10
} statetype;

statetype state, nextstate;

```

```
// state register
always_ff @(posedge clk, posedge reset)
```

4. Expected Operation

The table below shows the expected signal values for each pipeline step starting from step 3 (the first cycle after reset). Each I-type or R-type instruction takes 4 cycles (Fetch, Decode, Execute, ALUWB). A branch takes 3 cycles (Fetch, Decode, BEQ) and a load takes 5 cycles (Fetch, Decode, MemAdr, MemRead, MemWB). The Result column shows the value driven onto the Result bus at that cycle.

Step	PC	Instr	State	Result	Notes
3	00	00500113	S0: Fetch	4	Adr=PC=0, fetch IR. ALU=PC+4=4 OldPC←0, PC←4
4	04	""	S1: Decode	X	SrcA=OldPC=0, SrcB=ImmExt=5 ALUResult=5. $A \leftarrow x_0 = 0$
5	04	""	S8: Executel	X	SrcA=A=0, SrcB=Imm=5. ALUResult=0+5=5
6	04	""	S7: ALUWB	5	Result=ALUOut=5. $x_2 \leftarrow 5$
7	04	""	S0: Fetch	8	Adr=4, fetch 00C00193. PC←8
8	08	00C00193	S1: Decode	X	SrcA=OldPC=4, SrcB=ImmExt=12 ALUResult=16. $A \leftarrow x_0 = 0$
9	08	""	S8: Executel	X	SrcA=A=0, SrcB=Imm=12 ALUResult=0+12=12
10	08	""	S7: ALUWB	0C	Result=ALUOut=12 $x_3 \leftarrow 12$
11	08	""	S0: Fetch	0C	Adr=8, fetch FF718393 PC←0xC
12	0C	FF718393	S1: Decode	X	SrcA=OldPC=8, SrcB=ImmExt=-9 ALUResult=-1 $A \leftarrow x_3 = 12$
13	0C	""	S8: Executel	X	SrcA=A=12, SrcB=Imm=-9=FFFFFFF7 ALUResult=3
14	0C	""	S7: ALUWB	3	Result=ALUOut=3 $x_7 \leftarrow 3$
15	0C	""	S0: Fetch	10	Fetch 0023E233 (or) PC←0x10
16	10	0023E233	S1: Decode	X	$A \leftarrow x_7 = 3$, WriteData← $x_2 = 5$
17	10	""	S6: ExecuteR	X	SrcA=A=3, SrcB=WD=5 3 OR 5=7
18	10	""	S7: ALUWB	7	$x_4 \leftarrow 7$
19	10	""	S0: Fetch	14	Fetch 0041F2B3 (and) PC←0x14

20	14	0041F2B3	S1: Decode	X	$A \leftarrow x_3 = 12$, $\text{WriteData} \leftarrow x_4 = 7$
21	14	""	S6: ExecuteR	X	$12 \text{ AND } 7 = 4$
22	14	""	S7: ALUWB	4	$x_5 \leftarrow 4$
23	14	""	S0: Fetch	18	Fetch 004282B3 (add) $\text{PC} \leftarrow 0x18$
24	18	004282B3	S1: Decode	X	$A \leftarrow x_5 = 4$, $\text{WriteData} \leftarrow x_4 = 7$
25	18	""	S6: ExecuteR	X	$4 + 7 = 11$
26	18	""	S7: ALUWB	B	$x_5 \leftarrow 11$
27	18	""	S0: Fetch	1C	Fetch 02728863 $\text{PC} \leftarrow 0x1C$
28	1C	02728863	S1: Decode	X	$\text{OldPC} + \text{ImmExt} = 0x18 + 0x30 = 0x48$ (branch target)
29	1C	""	S10: BEQ	X	$x_5 - x_7 = 11 - 3 = 8$. Zero=0 Branch NOT taken!
30	1C	""	S0: Fetch	20	Fetch 0041A233 (slt). $\text{PC} \leftarrow 0x20$
31	20	0041A233	S1: Decode	X	$A \leftarrow x_3 = 12$, $\text{WriteData} \leftarrow x_4 = 7$
32	20	""	S6: ExecuteR	X	$12 < 7$? No $\rightarrow \text{SLT} = 0$.
33	20	""	S7: ALUWB	0	$x_4 \leftarrow 0$.
34	20	""	S0: Fetch	24	Fetch 00020463 (beq x_4, x_0). $\text{PC} \leftarrow 0x24$.
35	24	00020463	S1: Decode	X	$\text{OldPC} + \text{ImmExt} = 0x20 + 8 = 0x28$ (around).
36	24	""	S10: BEQ	28	$x_4 - x_0 = 0 - 0 = 0$. Zero=1. Branch TAKEN NOW $\text{PC} \leftarrow 0x28$
37	28	0023A233	S0: Fetch	2C	Fetch 0023A233 (slt x_4, x_7, x_2) $\text{PC} \leftarrow 0x2C$
38	2C	""	S1: Decode	X	$A \leftarrow x_7 = 3$, $\text{WriteData} \leftarrow x_2 = 5$
39	2C	""	S6: ExecuteR	X	$3 < 5$? Yes $\rightarrow \text{SLT} = 1$
40	2C	""	S7: ALUWB	1	$x_4 \leftarrow 1$
41	2C	""	S0: Fetch	30	Fetch 005203B3 (add x_7, x_4, x_5) $\text{PC} \leftarrow 0x30$
42	30	005203B3	S1: Decode	X	$A \leftarrow x_4 = 1$, $\text{WriteData} \leftarrow x_5 = 11$
43	30	""	S6: ExecuteR	X	$1 + 11 = 12$. $x_7 \leftarrow 12$ in next state

5. Simulation Waveform Analysis

I ran the testbench using the transcript in Questa. The simulation stopped at $t = 725$ ns and printed "Simulation succeeded" in the transcript. The sections below explain each waveform screenshot.

5.1 Transcript — Simulation Succeeded

The screenshot below shows the Questa transcript. The line "Simulation succeeded" confirms that the processor correctly wrote the value 25 to address 100. The simulation stopped at $t = 725$ ns as expected.

```
# Loading work.riscvmulti(fast)
# Loading work.controller(fast)
# Loading work.mainfsm(fast)
# Loading work.aludec(fast)
# Loading work.instrdec(fast)
# Loading work.datapath(fast)
# Loading work.floper(fast)
# Loading work.mux2(fast)
# Loading work.flopr(fast)
# Loading work.regfile(fast)
# Loading work.extend(fast)
# Loading work.mux3(fast)
# Loading work.alu(fast)
# Loading work.mem(fast)
VSIM 5> add wave sim:/testbench/clk
VSIM 6> add wave sim:/testbench/reset
VSIM 7> add wave -radix hex sim:/testbench/dut/rvmulti/dp/PC
VSIM 8> add wave -radix hex sim:/testbench/dut/rvmulti/dp/Instr
VSIM 9> add wave sim:/testbench/dut/rvmulti/c/fsm/state
VSIM 10> add wave -radix hex sim:/testbench/dut/rvmulti/dp/SrcA
VSIM 11> add wave -radix hex sim:/testbench/dut/rvmulti/dp/SrcB
VSIM 12> add wave -radix hex sim:/testbench/dut/rvmulti/dp/ALUResult
VSIM 13> add wave -radix hex sim:/testbench/DataAdr
VSIM 14> add wave -radix hex sim:/testbench/WriteData
VSIM 15> add wave sim:/testbench/MemWrite
VSIM 16> run 1500 ns
# Simulation succeeded
# ** Note: $stop : riscv_testbench.sv(30)
# Time: 725 ns Iteration: 1 Instance: /testbench
# Break in Module testbench at riscv_testbench.sv line 30
VSIM 17> wave zoom full
# {0 ns} {761 ns}
VSIM 18>]
```

Windows'u Etkinleştir
Windows'u etkinleştirmek için Ayarlar'a
gidin.

Figure 1: Questa transcript showing Simulation succeeded at $t = 725$ ns

5.2 Full Waveform Overview

The full waveform view (0 to 750 ns) shows all signals for the complete program execution. The MemWrite signal pulses twice: once at about 515 ns (first sw writing 7 to address 96) and again at about 720 ns (second sw writing 25 to address 100). All 32-bit signals are displayed in hexadecimal.

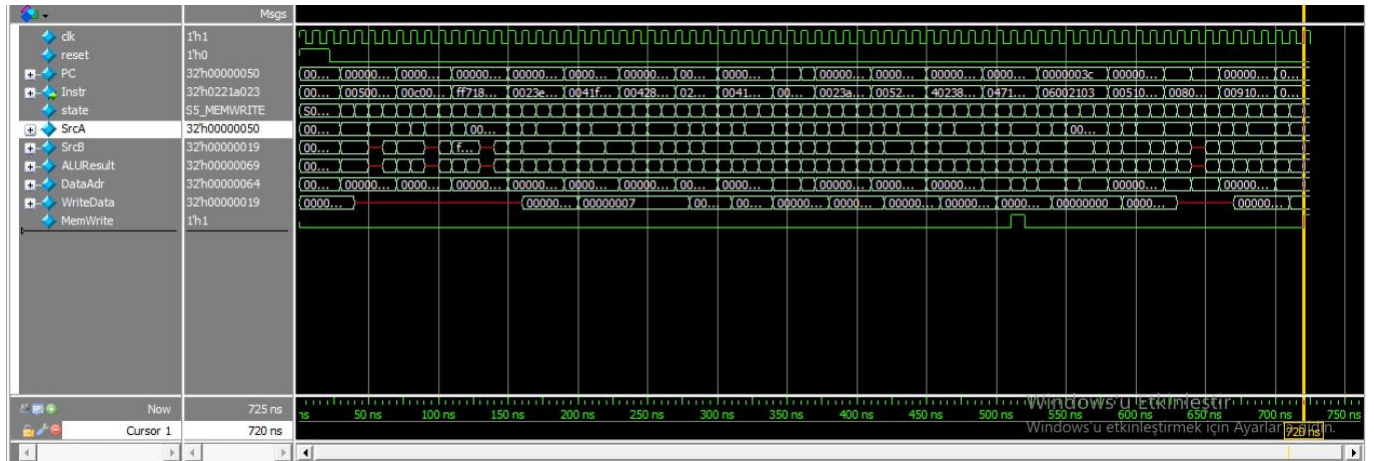


Figure 2: Full waveform overview (0 to 750 ns). MemWrite pulses visible at around 515 ns and 720 ns.

5.3 First Three Instructions (addi x2, addi x3, addi x7)

This zoomed view covers roughly 50 to 140 ns and shows the second and third addi instructions executing. The state sequence $S0_FETCH \rightarrow S1_DECODE \rightarrow S8_EXECUTEI \rightarrow S7_ALUWB$ repeats for each I-type instruction, confirming the 4-cycle execution pattern. The table below matches these waveform observations to my Table 1 predictions.



Figure 3: Zoomed waveform showing the first three addi instructions (steps 3-14 of Table 1)

State	PC	Instr	SrcA	SrcB	ALUResult
S8_EXECUTEI	00000004	00500113	00000000	00000005	00000005
S7_ALUWB	00000004	00500113	-	-	$x_2 \leftarrow 5$
S0_FETCH	00000004	loading	00000004	00000004	00000008
S1_DECODE	00000008	00C00193	00000004	0000000C	00000010

S8_EXECUTEI	00000008	00C00193	00000000	0000000C	0000000C
S7_ALUWB	00000008	00C00193	-	-	$x_3 \leftarrow 12$
S0_FETCH	00000008	loading	00000008	00000004	0000000C
S1_DECODE	0000000C	FF718393	00000008	FFFFFFF7	FFFFFFF7
S8_EXECUTEI	0000000C	FF718393	0000000C	FFFFFFF7	00000003
S7_ALUWB	0000000C	FF718393	-	-	$x_7 \leftarrow 3$

All values match my Table 1 predictions exactly. SrcB for the third instruction shows FFFFFFF7 which is the correct sign-extended encoding of -9.

5.4 First Store Instruction — sw x7, 84(x3)

This view covers about 492 to 536 ns and shows the sw x7, 84(x3) instruction executing. In state S2_MEMADR, SrcA holds $x_3=12=0x0C$ and SrcB holds ImmExt=84=0x54, so ALUResult = $0x0C + 0x54 = 0x60 = 96$. In state S5_MEMWRITE, DataAdr = 0x00000060 and WriteData = 0x00000007 ($x_7=7$) with MemWrite = 1. The testbench skips this write because DataAdr equals 96 and it is waiting for address 100.

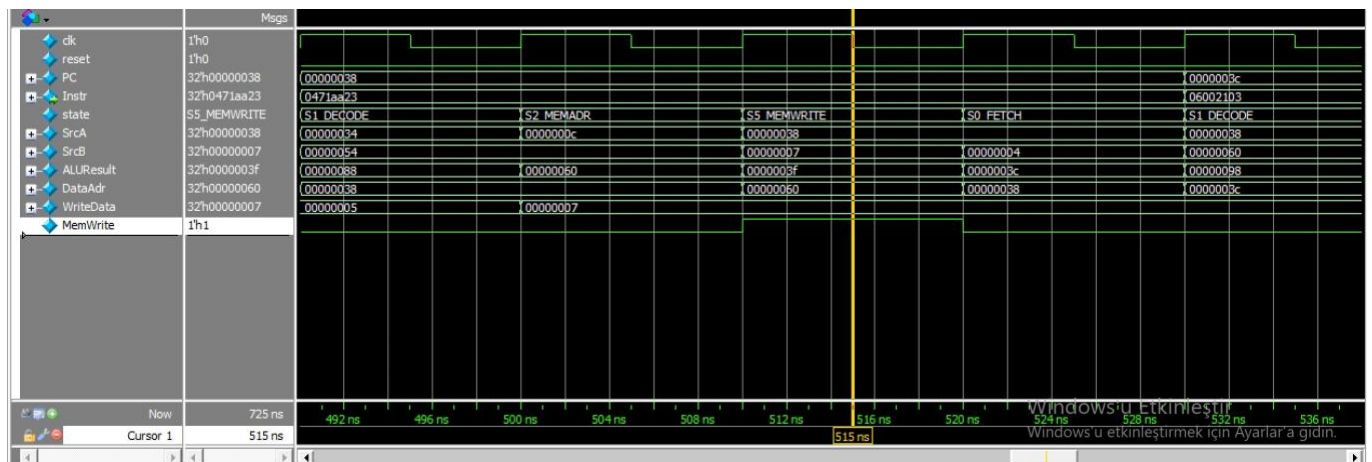


Figure 4: First sw instruction. See that DataAdr=0x60 (96), WriteData=0x07 (7), MemWrite=1 at ~515 ns

5.5 Final Store — sw x2, 0x20(x3) — SUCCESS

This zoomed view at 680 to 725 ns shows the final sw instruction that causes the testbench to print "Simulation succeeded". The state sequence is S7_ALUWB (completing add x2,x2,x9 which gives $x_2=25$) followed by S0_FETCH, S1_DECODE, S2_MEMADR and finally S5_MEMWRITE. In S2_MEMADR, SrcA = $x_3 = 0x00000044 = 68$ (the return address written by JAL) and SrcB = ImmExt = $0x20 = 32$, so ALUResult = $68 + 32 = 100 = 0x64$. In S5_MEMWRITE, DataAdr = 0x00000064 and WriteData = 0x00000019 = 25 with MemWrite = 1. This is the write that passes the testbench check.

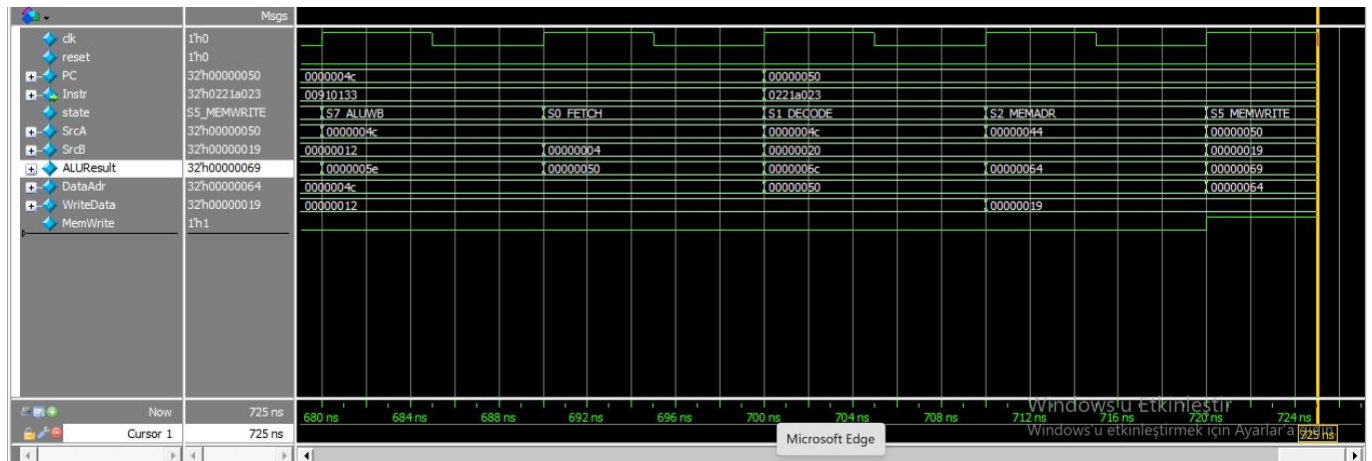


Figure 5: SUCCESS!! Final sw: DataAdr=0x64 (100), WriteData=0x19 (25), MemWrite=1 at 720 ns

State	PC	Instr	DataAdr	WriteData	MemWrite
S7_ALUWB	0000004C	00910133	0000004C	00000019	0
S0_FETCH	0000004C	loading	0000004C	00000019	0
S1_DECODE	00000050	0221A023	00000050	00000019	0
S2_MEMADR	00000050	0221A023	00000050	00000019	0
S5_MEMWRITE	00000050	0221A023	00000064	00000019	1

The highlighted row (S5_MEMWRITE) is the cycle that triggers the testbench success condition: DataAdr = 100 and WriteData = 25.